

Playbook: Build a similar full-stack app with Cursor

Share this with someone who wants to ship a **single Node server** that serves both a **web UI** and a **REST API**, backed by a **real database**, using **Cursor** as the editor and **Git + GitHub** for code—and optionally deploy to a VPS or cloud app platform later.

This mirrors the *shape* of many production ERP-style apps (one Express process, `/api/*`, SPA/static assets, Prisma + Postgres). It is **not** a copy of any proprietary codebase.

1. What you are aiming for

Piece	Role
Backend	Node.js + Express (or similar): JSON APIs under <code>/api/...</code> , auth, validation.
Frontend	Browser UI (React or similar): built files served as static assets or embedded entry.
Database	PostgreSQL in production; often SQLite or local Postgres while learning.
ORM	Prisma is a solid default (schema in code, migrations, type-safe client).
Hosting (later)	One VM (e.g. DigitalOcean droplet) with Nginx + process manager (PM2), or a PaaS that runs Node.

One mental model: the browser talks **HTTPS** to **one server process**. That process answers `/api/*` from the database and serves HTML/JS/CSS for everything else.

2. Install once on your machine

- **Node.js** LTS (18 or 20+) — nodejs.org
- **Git** — git-scm.com
- **Cursor** — cursor.com (sign in; optional Pro if you use AI heavily)
- **PostgreSQL** locally *optional* for early experiments (Docker or native installer)

Verify:

```
node -v
npm -v
git --version
```

3. Create a new project (minimal spine)

From an empty folder:

```
mkdir my-app && cd my-app
npm init -y
npm install express dotenv
```

Add `server.js` (bare minimum):

```
import 'dotenv/config'
import express from 'express'
const app = express()
app.use(express.json())
app.get('/api/health', (_req, res) => res.json({ ok: true }))
app.use(express.static('public'))
const port = process.env.PORT || 3000
app.listen(port, () => console.log(`http://localhost:${port}`))
```

Add `public/index.html` with a single `<h1>` so you prove static hosting works.

```
node server.js
# Visit http://localhost:3000 - UI
# Visit http://localhost:3000/api/health - JSON
```

Why this order: proves **API + static** from one process before adding React or a database.

4. Add Prisma + PostgreSQL

```
npm install prisma @prisma/client
npx prisma init
```

- Edit `prisma/schema.prisma`: set `provider = "postgresql"` (or `sqlite` for zero-setup local dev).
- Set `DATABASE_URL` in `.env` (never commit real secrets—use `.env.example` for dummy values).

```
npx prisma migrate dev --name init
```

Create a tiny API route that reads/writes the DB (e.g. `GET /api/items`). Keep handlers small and test with curl or the browser.

5. Add a real frontend workflow

Pick **one** approach and stay consistent:

A. Bundled SPA (Vite + React) — `npm create vite@latest client` then build into `public/` or `dist/` and point Express at that folder.

B. JSX compiled to JS — build step outputs bundles your `index.html` loads (same idea: built assets under a folder Express serves).

Cursor tip: Ask the AI to “add Vite React in a `client/` folder and serve `dist` from Express in production”—then iterate file by file.

6. Use Cursor effectively (without drowning)

Practice	Why
<code>.cursorrules</code> or project rules	Tell the AI which folders are canonical (<code>src/</code> vs experiments), naming, and “do not edit X”.
Small asks	One endpoint or one component per task reduces wrong edits.
@ files	Reference specific files in chat so answers match your tree.
Run terminal yourself or via agent	<code>npm run dev</code> , <code>npx prisma studio</code> , fix errors in the loop.

7. Git + GitHub (non-negotiable for collaboration)

```
git init
echo "node_modules/
.env
.DS_Store" >> .gitignore
git add .
git commit -m "Initial Express + health route"
```

Create an empty repo on GitHub, then:

```
git remote add origin git@github.com:YOUR_USER/YOUR_REPO.git
git branch -M main
git push -u origin main
```

Branch habit: `feature/...` for work, open PRs if more than one person touches the repo.

8. Environment variables (production-grade habit)

- **Development:** `.env` local only, gitignored.
 - **Production:** set vars on the host (droplet `.env` , or platform UI)—never bake secrets into the image/repo.
 - Typical vars: `DATABASE_URL` , `JWT_SECRET` , `NODE_ENV=production` , `PORT` .
-

9. Deploy (high level—same “family” as a serious ERP)

1. **Build** on the server or in CI: `npm ci` , `npm run build` , `npx prisma migrate deploy` .
2. **Run** Node behind **Nginx** (HTTPS) or use a platform that terminates TLS for you.
3. **Supervise** with **PM2** or `systemd` so crashes restart.
4. Point **DNS** (A record) at your server when you have a domain.

Your friend does **not** need every bell-whistle on day one—**health check + DB + HTTPS** is enough to call it “real.”

10. Checklist before showing it to users

- ☐ `/api/health` (or similar) returns 200 on the server.
 - ☐ Database migrations applied in production.
 - ☐ HTTPS works (Let's Encrypt via Nginx or host-managed certs).
 - ☐ Secrets only in env—not in Git.
 - ☐ Basic backup story for Postgres (managed DB backups or dumps).
-

11. If they get stuck

- **503 / timeouts on APIs:** often DB URL, firewall, or connection limits—not “React bugs.”
 - **Blank page after deploy:** wrong `PORT`, wrong static path, or build output not uploaded.
 - **Works locally, fails in prod:** compare `NODE_ENV`, env vars, and CORS/origin settings.
-

Summary one-liner

Cursor helps you write and refactor faster; your architecture is still “Express API + static SPA + Postgres,” committed to GitHub, deployed as one Node process behind HTTPS.

You can duplicate this file into your friend's repo or send it as-is. Adjust section 9 to match whichever cloud provider they choose.